Deep Reinforcement Learning

Unit 6: Advantage Actor-Critic (A2C)

Hugging Face Deep RL Course

Complete MDX-to-LaTeX study edition

Source: Hugging Face Deep RL Course (original English text).

Contents

1 Introduction **2**

2 The Problem of Variance in Reinforce **2**

3 Advantage Actor-Critic (A2C) **4**
- 3.1 Reducing variance with Actor-Critic methods . . . 4
- 3.2 The Actor-Critic Process . . . 5
- 3.3 Adding Advantage in Actor-Critic (A2C) . . . 7
- 3.4 Advantage Actor Critic (A2C) using Robotics Simulations with Panda-Gym . . . 8
- 3.5 Unit 6: Advantage Actor Critic (A2C) using Robotics Simulations with Panda-Gym . . . 8
- 3.6 Let's train our first robots . . . 9

4 Quiz **16**
- 4.1 Quiz solutions and explanations . . . 17

5 Conclusion **18**

6 Additional Readings **19**
- 6.1 Bias-variance tradeoff in Reinforcement Learning . . . 19
- 6.2 Advantage Functions . . . 19
- 6.3 Actor Critic . . . 19

1 Introduction



Figure 1: Introduction to Actor-Critic methods: official course cover [original figure]

In unit 4, we learned about our first Policy-Based algorithm called **Reinforce**.

In Policy-Based methods, **we aim to optimize the policy directly without using a value function**. More precisely, Reinforce is part of a subclass of *Policy-Based Methods* called *Policy-Gradient methods*. This subclass optimizes the policy directly by **estimating the weights of the optimal policy using Gradient Ascent**.

We saw that Reinforce worked well. However, because we use Monte-Carlo sampling to estimate return (we use an entire episode to calculate the return), **we have significant variance in policy gradient estimation**.

Remember that the policy gradient estimation is **the direction of the steepest increase in return**. In other words, how to update our policy weights so that actions that lead to good returns have a higher probability of being taken. The Monte Carlo variance, which we will further study in this unit, **leads to slower training since we need a lot of samples to mitigate it**.

So today we'll study **Actor-Critic methods**, a hybrid architecture combining value-based and Policy-Based methods that helps to stabilize the training by reducing the variance using:

- *An Actor* that controls **how our agent behaves** (Policy-Based method)
- *A Critic* that measures **how good the taken action is** (Value-Based method)

We'll study one of these hybrid methods, Advantage Actor Critic (A2C), **and train our agent using Stable-Baselines3 in robotic environments**. We'll train:

- A robotic arm to move to the correct position.

Sound exciting? Let's get started!

2 The Problem of Variance in Reinforce

In Reinforce, we want to **increase the probability of actions in a trajectory proportionally to how high the return is**.

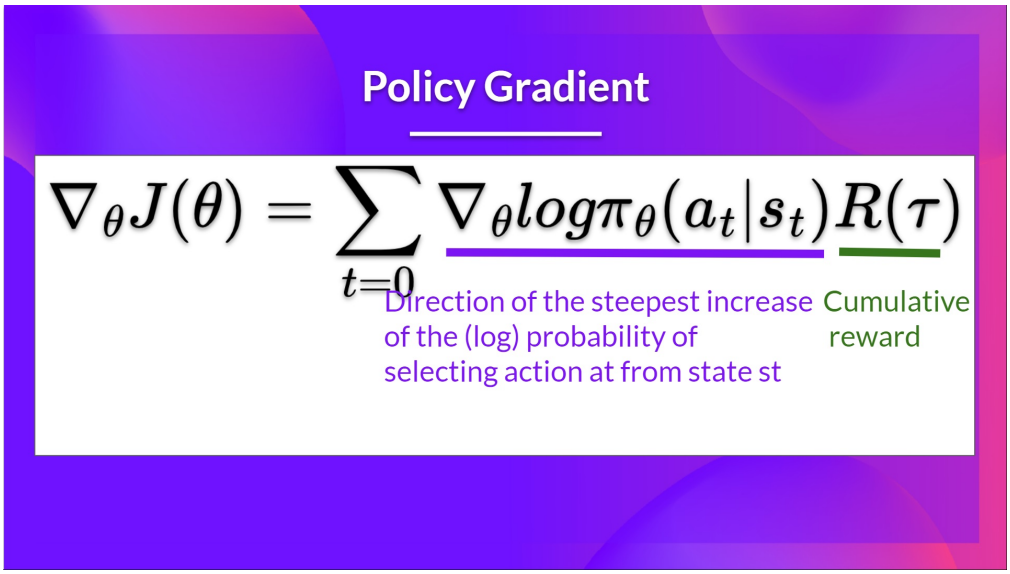


Figure 2: The REINFORCE policy-gradient update [original figure]

- If the **return is high**, we will **push up** the probabilities of the (state, action) combinations.
- Otherwise, if the **return is low**, it will **push down** the probabilities of the (state, action) combinations.

This return $R(\tau)$ is calculated using a *Monte-Carlo sampling*. We collect a trajectory and calculate the discounted return, **and use this score to increase or decrease the probability of every action taken in that trajectory**. If the return is good, all actions will be "reinforced" by increasing their likelihood of being taken.

$R(\tau) = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ...$

The advantage of this method is that **it's unbiased. Since we're not estimating the return**, we use only the true return we obtain.

Given the stochasticity of the environment (random events during an episode) and stochasticity of the policy, **trajectories can lead to different returns, which can lead to high variance**. Consequently, the same starting state can lead to very different returns. Because of this, **the return starting at the same state can vary significantly across episodes**.

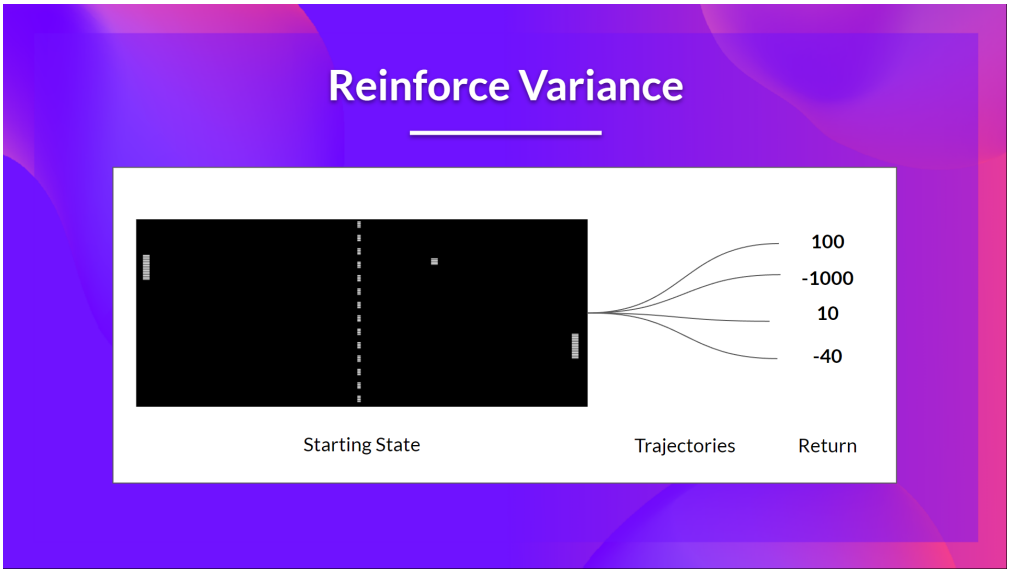


Figure 3: Variance of Monte Carlo returns across trajectories [original figure]

The solution is to mitigate the variance by **using a large number of trajectories, hoping that the variance introduced in any one trajectory will be reduced in aggregate and provide**

a "true" estimation of the return.

However, increasing the batch size significantly **reduces sample efficiency**. So we need to find additional mechanisms to reduce the variance.

If you want to dive deeper into the question of variance and bias tradeoff in Deep Reinforcement Learning, you can check out these two articles:

- Making Sense of the Bias / Variance Trade-off in (Deep) Reinforcement Learning
- Bias-variance Tradeoff in Reinforcement Learning
- High Variance in Policy gradients ***

3 Advantage Actor-Critic (A2C)

3.1 Reducing variance with Actor-Critic methods

The solution to reducing the variance of the Reinforce algorithm and training our agent faster and better is to use a combination of Policy-Based and Value-Based methods: *the Actor-Critic method*.

To understand the Actor-Critic, imagine you're playing a video game. You can play with a friend that will provide you with some feedback. You're the Actor and your friend is the Critic.

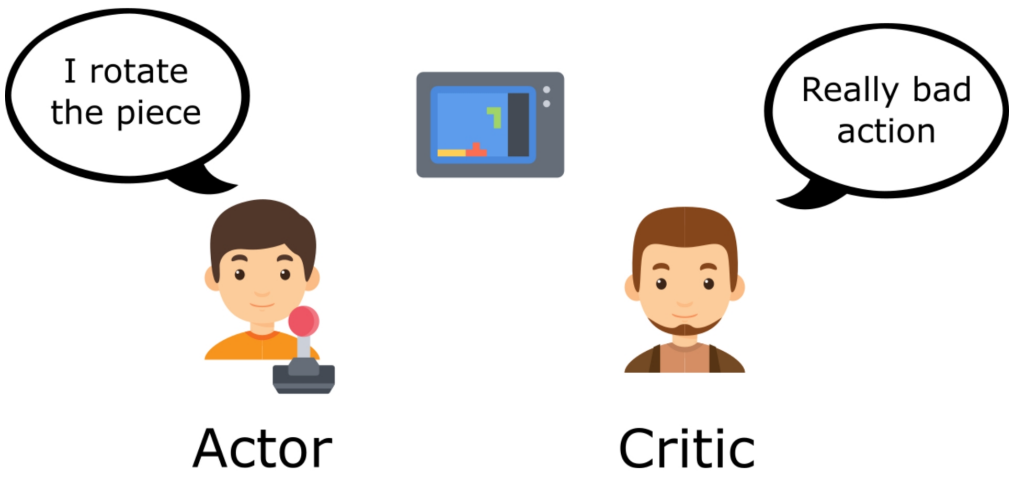


Figure 4: Actor and Critic: policy and value-function feedback [original figure]

You don't know how to play at the beginning, **so you try some actions randomly**. The Critic observes your action and **provides feedback**.

Learning from this feedback, **you'll update your policy and be better at playing that game.**

On the other hand, your friend (Critic) will also update their way to provide feedback so it can be better next time.

This is the idea behind Actor-Critic. We learn two function approximations:

- *A policy* that **controls how our agent acts**: $\pi_\theta(s)$
- *A value function* to assist the policy update by measuring how good the action taken is: $\hat{q}_w(s, a)$

3.2 The Actor-Critic Process

Now that we have seen the Actor Critic's big picture, let's dive deeper to understand how the Actor and Critic improve together during the training.

As we saw, with Actor-Critic methods, there are two function approximations (two neural networks):

- *Actor*, a **policy function** parameterized by theta: $\pi_\theta(s)$
- *Critic*, a **value function** parameterized by w: $\hat{q}_w(s, a)$

Let's see the training process to understand how the Actor and Critic are optimized:

- At each timestep, t, we get the current state S_t from the environment and **pass it as input through our Actor and Critic**.
- Our Policy takes the state and **outputs an action** A_t.

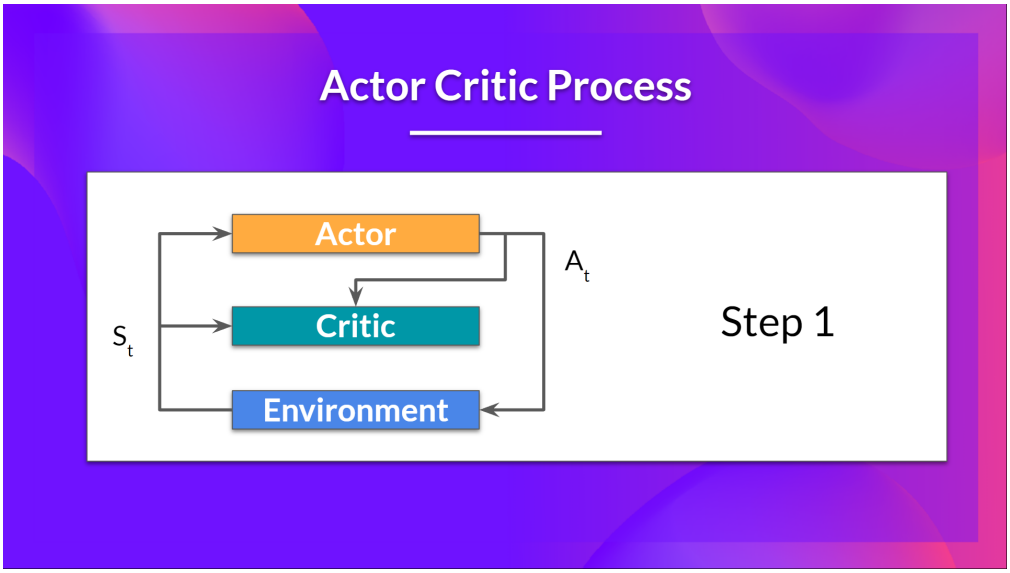


Figure 5: Actor-Critic training process: actor selects an action [original figure]

- The Critic takes that action also as input and, using S_t and A_t, **computes the value of taking that action at that state: the Q-value**.

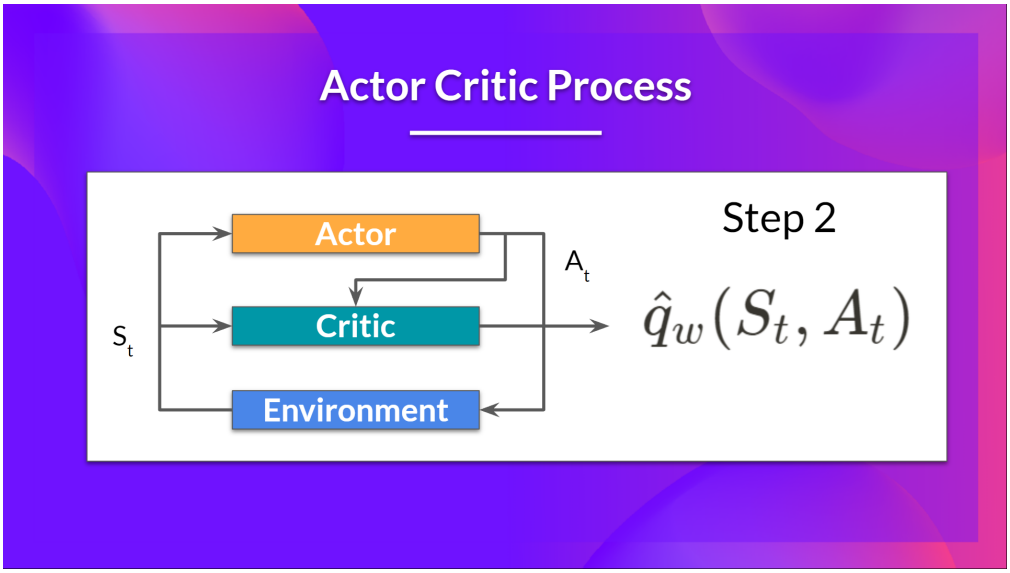


Figure 6: Actor-Critic process: critic evaluates the state-action pair [original figure]

- The action A_t performed in the environment outputs a new state S_{t+1} and a reward R_{t+1} .



Figure 7: Actor-Critic process: the environment returns reward and next state [original figure]

- The Actor updates its policy parameters using the Q value.

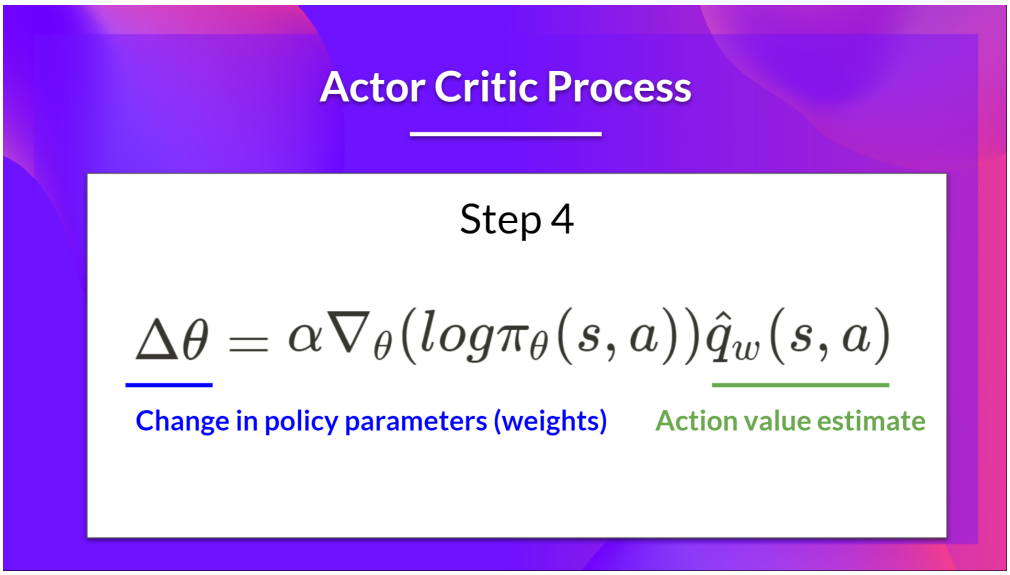


Figure 8: Actor-Critic process: updating the actor using the Q-value [original figure]

- Thanks to its updated parameters, the Actor produces the next action to take at A_{t+1} given the new state S_{t+1}.
- The Critic then updates its value parameters.

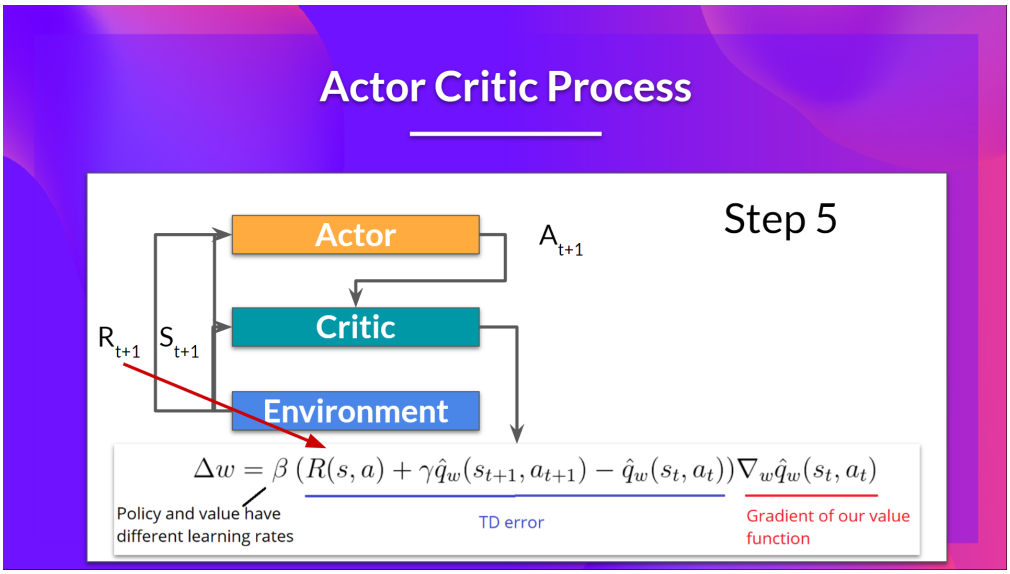


Figure 9: Actor-Critic process: updating the critic [original figure]

3.3 Adding Advantage in Actor-Critic (A2C)

We can stabilize learning further by **using the Advantage function as Critic instead of the Action value function**.

The idea is that the Advantage function calculates the relative advantage of an action compared to the others possible at a state: **how taking that action at a state is better compared to the average value of the state**. It's subtracting the mean value of the state from the state action pair:

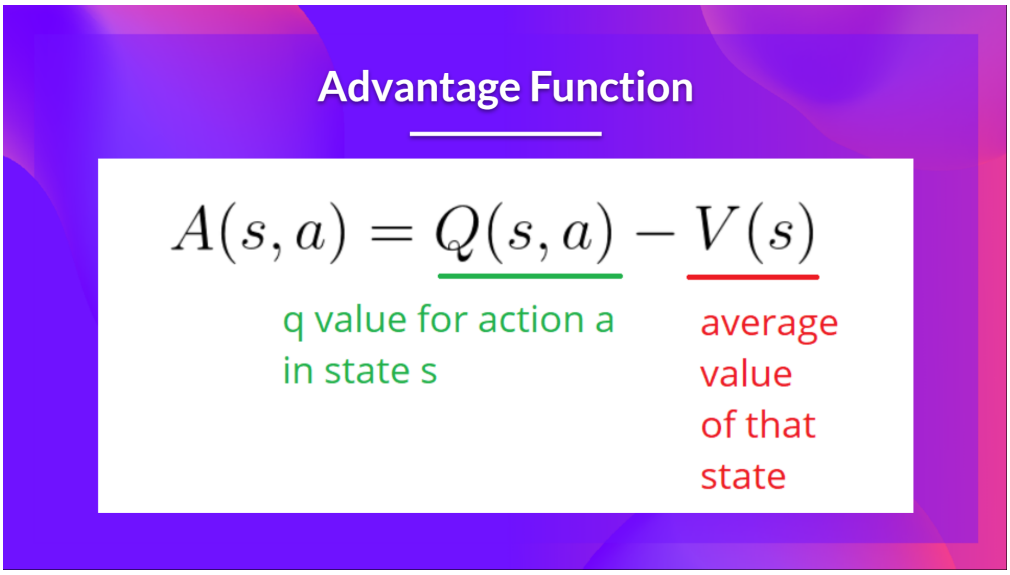


Figure 10: Definition of the advantage function [original figure]

In other words, this function calculates **the extra reward we get if we take this action at that state compared to the mean reward we get at that state**.

The extra reward is what's beyond the expected value of that state.

- If A(s,a) $>$ 0: our gradient is **pushed in that direction**.
- If A(s,a) $<$ 0 (our action does worse than the average value of that state), **our gradient is pushed in the opposite direction**.

The problem with implementing this advantage function is that it requires two value functions — $Q(s,a)$ and $V(s)$. Fortunately, **we can use the TD error as a good estimator of the advantage function.**

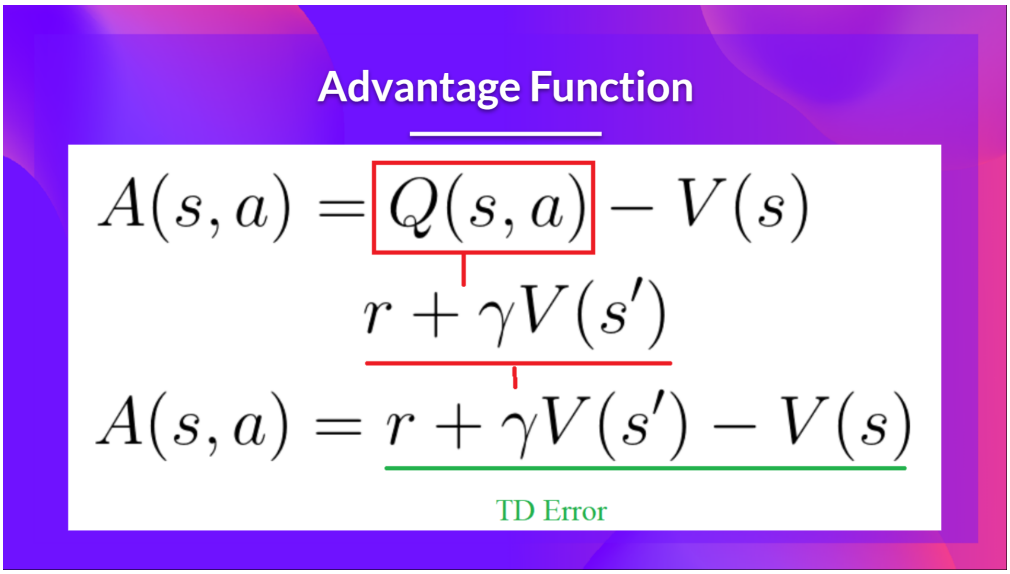


Figure 11: TD error as an advantage estimator [original figure]

3.4 Advantage Actor Critic (A2C) using Robotics Simulations with Panda-Gym

Official Unit 6 practical notebook (Google Colab)

Community help

Now that you've studied the theory behind Advantage Actor Critic (A2C), **you're ready to train your A2C agent** using Stable-Baselines3 in a robotic environment. And train a:

- A robotic arm to move to the correct position.

We're going to use

- panda-gym

To validate this hands-on for the certification process, you need to push your two trained models to the Hub and get the following results:

- `PandaReachDense-v3` get a result of >= -3.5.

To find your result, go to the leaderboard and find your model, **the result = mean_reward - std of reward**

For more information about the certification process, check this section https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

To start the hands-on click on Open In Colab button :

Open in Google Colab

3.5 Unit 6: Advantage Actor Critic (A2C) using Robotics Simulations with Panda-Gym

Environments:

- Panda-Gym

RL-Library:

- Stable-Baselines3

We're constantly trying to improve our tutorials, so **if you find some issues in this notebook**, please open an issue on the GitHub Repo.

Objectives of this notebook

At the end of the notebook, you will:

- Be able to use **Panda-Gym**, the environment library.
- Be able to **train robots using A2C**.
- Understand why **we need to normalize the input**.
- Be able to **push your trained agent and the code to the Hub** with a nice video replay and an evaluation score .

Prerequisites

Before diving into the notebook, you need to:

Study Actor-Critic methods by reading Unit 6

3.6 Let's train our first robots

Set the GPU

- To **accelerate the agent's training, we'll use a GPU**. To do that, go to `Runtime > Change Runtime type`

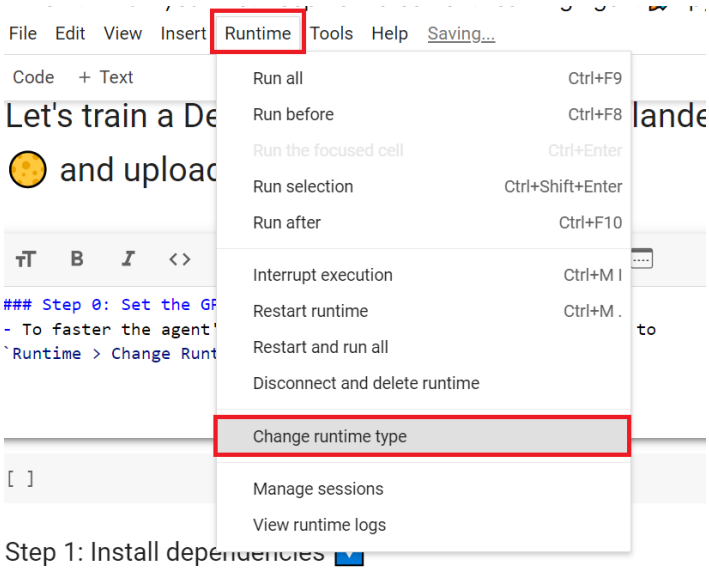


Figure 12: Google Colab: selecting runtime settings [original figure]

- `Hardware Accelerator > GPU`

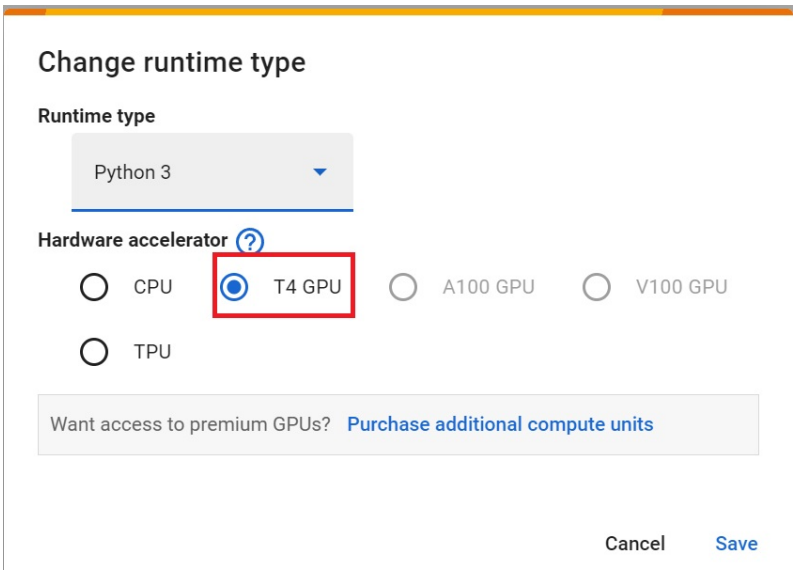


Figure 13: Google Colab: enabling GPU acceleration [original figure]

Create a virtual display

During the notebook, we'll need to generate a replay video. To do so, with colab, **we need to have a virtual screen to be able to render the environment** (and thus record the frames).

The following cell will install the librairies and create and run a virtual screen

```
%%capture
!apt install python-opengl
!apt install ffmpeg
!apt install xvfb
!pip3 install pyvirtualdisplay
```

```
# Virtual display
from pyvirtualdisplay import Display

virtual_display = Display(visible=0, size=(1400, 900))
virtual_display.start()
```

Install dependencies We'll install multiple ones:

- `gymnasium`
- `panda-gym`: Contains the robotics arm environments.
- `stable-baselines3`: The SB3 deep reinforcement learning library.
- `huggingface_sb3`: Additional code for Stable-baselines3 to load and upload models from the Hugging Face Hub.
- `huggingface_hub`: Library allowing anyone to work with the Hub repositories.

```
!pip install stable-baselines3[extra]
!pip install gymnasium
!pip install huggingface_sb3
!pip install huggingface_hub
!pip install panda_gym
```

Import the packages

```
import os

import gymnasium as gym
import panda_gym

from huggingface_sb3 import load_from_hub, package_to_hub

from stable_baselines3 import A2C
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize
from stable_baselines3.common.env_util import make_vec_env

from huggingface_hub import notebook_login
```

PandaReachDense-v3

The agent we're going to train is a robotic arm that needs to do controls (moving the arm and using the end-effector).

In robotics, the *end-effector* is the device at the end of a robotic arm designed to interact with the environment.

In `PandaReach`, the robot must place its end-effector at a target position (green ball).

We're going to use the dense version of this environment. It means we'll get a *dense reward function* that **will provide a reward at each timestep** (the closer the agent is to completing the task, the higher the reward). Contrary to a *sparse reward function* where the environment **return a reward if and only if the task is completed**.

Also, we're going to use the *End-effector displacement control*, it means the **action corresponds to the displacement of the end-effector**. We don't control the individual motion of each joint (joint control).

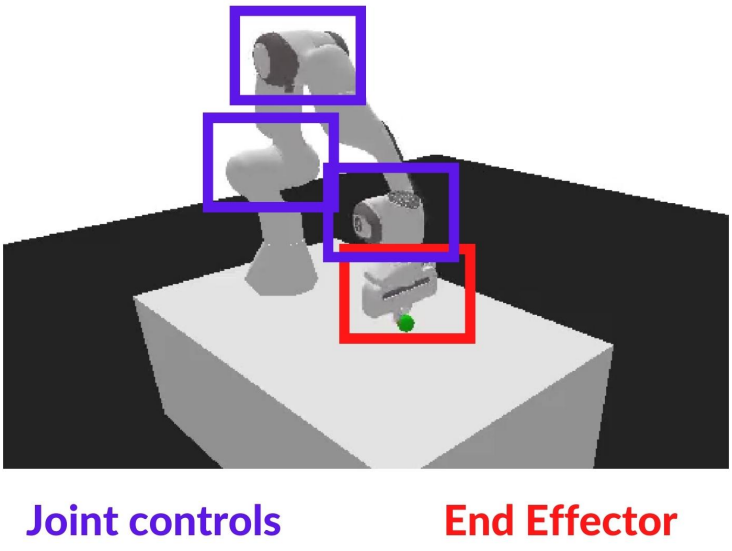


Figure 14: PandaReach robotic-arm environment [original figure]

This way **the training will be easier**.

Create the environment

The environment In `PandaReachDense-v3` the robotic arm must place its end-effector at a target position (green ball).

```
env_id = "PandaReachDense-v3"

# Create the env
env = gym.make(env_id)

# Get the state space and action space
s_size = env.observation_space.shape
a_size = env.action_space
```

```
print("_____OBSERVATION SPACE_____ \n")
print("The State Space is: ", s_size)
print("Sample observation", env.observation_space.sample()) # Get a random observation
```

The observation space **is a dictionary with 3 different elements**:

- `achieved_goal`: (x,y,z) position of the goal.
- `desired_goal`: (x,y,z) distance between the goal position and the current object position.
- `observation`: position (x,y,z) and velocity of the end-effector (vx, vy, vz).

Given it's a dictionary as observation, **we will need to use a MultiInputPolicy policy instead of MlpPolicy**.

```
print("\n _____ACTION SPACE_____ \n")
print("The Action Space is: ", a_size)
print("Action Space Sample", env.action_space.sample()) # Take a random action
```

The action space is a vector with 3 values:

- Control x, y, z movement

Normalize observation and rewards A good practice in reinforcement learning is to normalize input features.

For that purpose, there is a wrapper that will compute a running average and standard deviation of input features.

We also normalize rewards with this same wrapper by adding `norm_reward = True`

You should check the documentation to fill this cell

```
env = make_vec_env(env_id, n_envs=4)

# Adding this wrapper to normalize the observation and the reward
env = # TODO: Add the wrapper
```

Solution

```
env = make_vec_env(env_id, n_envs=4)

env = VecNormalize(env, norm_obs=True, norm_reward=True, clip_obs=10.)
```

Create the A2C Model For more information about A2C implementation with StableBaselines3 check: https://stable-baselines3.readthedocs.io/en/master/modules/a2c.html#notes

To find the best parameters I checked the official trained agents by Stable-Baselines3 team.

```
model = # Create the A2C model and try to find the best parameters
```

Solution

```
model = A2C(policy = "MultiInputPolicy",
            env = env,
            verbose=1)
```

Train the A2C agent

- Let's train our agent for 1,000,000 timesteps, don't forget to use GPU on Colab. It will take approximately ~25-40min

```
model.learn(1_000_000)
```

```
# Save the model and  VecNormalize statistics when saving the agent
model.save("a2c-PandaReachDense-v3")
env.save("vec_normalize.pkl")
```

Evaluate the agent

- Now that's our agent is trained, we need to **check its performance**.
- Stable-Baselines3 provides a method to do that: `evaluate_policy`

```
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize

# Load the saved statistics
eval_env = DummyVecEnv([lambda: gym.make("PandaReachDense-v3")])
eval_env = VecNormalize.load("vec_normalize.pkl", eval_env)

# We need to override the render_mode
eval_env.render_mode = "rgb_array"

#  do not update them at test time
eval_env.training = False
# reward normalization is not needed at test time
eval_env.norm_reward = False

# Load the agent
model = A2C.load("a2c-PandaReachDense-v3")

mean_reward, std_reward = evaluate_policy(model, eval_env)

print(f"Mean reward = {mean_reward:.2f} +/- {std_reward:.2f}")
```

Publish your trained model on the Hub Now that we saw we got good results after the training, we can publish our trained model on the Hub with one line of code.

The libraries documentation https://github.com/huggingface/huggingface_sb3/tree/main#hugging-face--x-stable-baselines3-v20

By using `package_to_hub`, as we already mentionned in the former units, **you evaluate, record a replay, generate a model card of your agent and push it to the hub**.

This way:

- You can **showcase our work**
- You can **visualize your agent playing**
- You can **share with the community an agent that others can use**
- You can **access a leaderboard to see how well your agent is performing compared to your classmates** https : / / huggingface . co / spaces / huggingface-projects / Deep-Reinforcement-Learning-Leaderboard

To be able to share your model with the community there are three more steps to follow:

1. (If it's not already done) create an account to HF -> https://huggingface.co/join
2. Sign in and then, you need to store your authentication token from the Hugging Face website.

- Create a new token (https://huggingface.co/settings/tokens) **with write role**

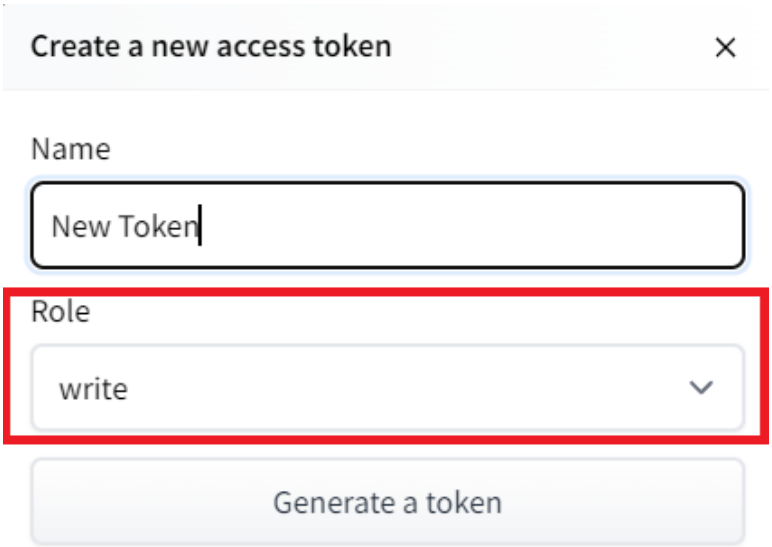


Figure 15: Creating a Hugging Face write-access token [original figure]

- Copy the token
- Run the cell below and paste the token

```
notebook_login()
!git config --global credential.helper store
```

If you don't want to use a Google Colab or a Jupyter Notebook, you need to use this command instead: `huggingface-cli login`

3. We're now ready to push our trained agent to the Hub using `package_to_hub()` function. For this environment, **running this cell can take approximately 10min**

```
from huggingface_sb3 import package_to_hub

package_to_hub(
    model=model,
    model_name=f"a2c-{env_id}",
    model_architecture="A2C",
    env_id=env_id,
    eval_env=eval_env,
    repo_id=f"ThomasSimonini/a2c-{env_id}", # Change the username
```

```
    commit_message="Initial commit",
)
```

Some additional challenges

The best way to learn **is to try things by your own**! Why not trying `PandaPickAndPlace-v3`?

If you want to try more advanced tasks for panda-gym, you need to check what was done using **TQC or SAC** (a more sample-efficient algorithm suited for robotics tasks). In real robotics, you'll use a more sample-efficient algorithm for a simple reason: contrary to a simulation **if you move your robotic arm too much, you have a risk of breaking it**.

PandaPickAndPlace-v1 (this model uses the v1 version of the environment): https://huggingface.co/sb3/tqc-PandaPickAndPlace-v1

And don't hesitate to check panda-gym documentation here: https://panda-gym.readthedocs.io/en/latest/usage/train_with_sb3.html

We provide you the steps to train another agent (optional):

1. Define the environment called "PandaPickAndPlace-v3"
2. Make a vectorized environment
3. Add a wrapper to normalize the observations and rewards. Check the documentation
4. Create the A2C Model (don't forget verbose=1 to print the training logs).
5. Train it for 1M Timesteps
6. Save the model and VecNormalize statistics when saving the agent
7. Evaluate your agent
8. Publish your trained model on the Hub with `package_to_hub`

Solution (optional)

```
# 1 - 2
env_id = "PandaPickAndPlace-v3"
env = make_vec_env(env_id, n_envs=4)

# 3
env = VecNormalize(env, norm_obs=True, norm_reward=True, clip_obs=10.)

# 4
model = A2C(policy = "MultiInputPolicy",
            env = env,
            verbose=1)
# 5
model.learn(1_000_000)
```

```
# 6
model_name = "a2c-PandaPickAndPlace-v3";
model.save(model_name)
env.save("vec_normalize.pkl")

# 7
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize

# Load the saved statistics
eval_env = DummyVecEnv([lambda: gym.make("PandaPickAndPlace-v3")])
eval_env = VecNormalize.load("vec_normalize.pkl", eval_env)

#  do not update them at test time
eval_env.training = False
# reward normalization is not needed at test time
eval_env.norm_reward = False
```

```
# Load the agent
model = A2C.load(model_name)

mean_reward, std_reward = evaluate_policy(model, eval_env)

print(f"Mean reward = {mean_reward:.2f} +/- {std_reward:.2f}")

# 8
package_to_hub(
    model=model,
    model_name=f"a2c-{env_id}",
    model_architecture="A2C",
    env_id=env_id,
    eval_env=eval_env,
    repo_id=f"ThomasSimonini/a2c-{env_id}", # TODO: Change the username
    commit_message="Initial commit",
)
```

See you on Unit 7!

Keep learning, stay awesome

4 Quiz

The best way to learn and to avoid the illusion of competence **is to test yourself.** This will help you to find **where you need to reinforce your knowledge**.

Q1: Which of the following interpretations of bias-variance tradeoff is the most accurate in the field of Reinforcement Learning?

- **A.** The bias-variance tradeoff reflects how my model is able to generalize the knowledge to previously tagged data we give to the model during training time.
- **B.** The bias-variance tradeoff reflects how well the reinforcement signal reflects the true reward the agent should get from the enviromment

Q2: Which of the following statements are true, when talking about models with bias and/or variance in RL?

- **A.** An unbiased reward signal returns rewards similar to the real / expected ones from the environment
- **B.** A biased reward signal returns rewards similar to the real / expected ones from the environment
- **C.** A reward signal with high variance has much noise in it and gets affected by, for example, stochastic (non constant) elements in the environment
- **D.** A reward signal with low variance has much noise in it and gets affected by, for example, stochastic (non constant) elements in the environment

Q3: Which of the following statements are true about Monte Carlo method?

- **A.** It's a sampling mechanism, which means we don't analyze all the possible states, but a sample of those
- **B.** It's very resistant to stochasticity (random elements in the trajectory)
- **C.** To reduce the impact of stochastic elements in Monte Carlo, we take `n` strategies and average them, reducing their individual impact

Q4: How would you describe, with your own words, the Actor-Critic Method (A2C)?

Q5: Which of the following statements are true about the Actor-Critic Method?

- **A.** The Critic does not learn any function during the training process

- **B.** The Actor learns a policy function, while the Critic learns a value function
- **C.** It adds resistance to stochasticity and reduces high variance

Q6: What is `Advantage` in the A2C method?

Congrats on finishing this Quiz , if you missed some elements, take time to read the chapter again to reinforce () your knowledge.

4.1 Quiz solutions and explanations

Q1 solution

Correct answer(s): B

A. The bias-variance tradeoff reflects how my model is able to generalize the knowledge to previously tagged data we give to the model during training time. Explanation: This is the traditional bias-variance tradeoff in Machine Learning. In our specific case of Reinforcement Learning, we don't have previously tagged data, but only a reward signal.

B. The bias-variance tradeoff reflects how well the reinforcement signal reflects the true reward the agent should get from the enviromment

Q2 solution

Correct answer(s): A, C

A. An unbiased reward signal returns rewards similar to the real / expected ones from the environment

B. A biased reward signal returns rewards similar to the real / expected ones from the environment Explanation: If a reward signal is biased, it means the reward signal we get differs from the real reward we should be getting from an environment

C. A reward signal with high variance has much noise in it and gets affected by, for example, stochastic (non constant) elements in the environment

D. A reward signal with low variance has much noise in it and gets affected by, for example, stochastic (non constant) elements in the environment Explanation: If a reward signal has low variance, then it's less affected by the noise of the environment and produce similar values regardless the random elements in the environment

Q3 solution

Correct answer(s): A, C

A. It's a sampling mechanism, which means we don't analyze all the possible states, but a sample of those

B. It's very resistant to stochasticity (random elements in the trajectory) Explanation: Monte Carlo randomly estimates everytime a sample of trajectories. However, even same trajectories can have different reward values if they contain stochastic elements

C. To reduce the impact of stochastic elements in Monte Carlo, we take `n` strategies and average them, reducing their individual impact

Q4 solution

The idea behind Actor-Critic is that we learn two function approximations:

1. A `policy` that controls how our agent acts (π)
2. A `value` function to assist the policy update by measuring how good the action taken is (q)

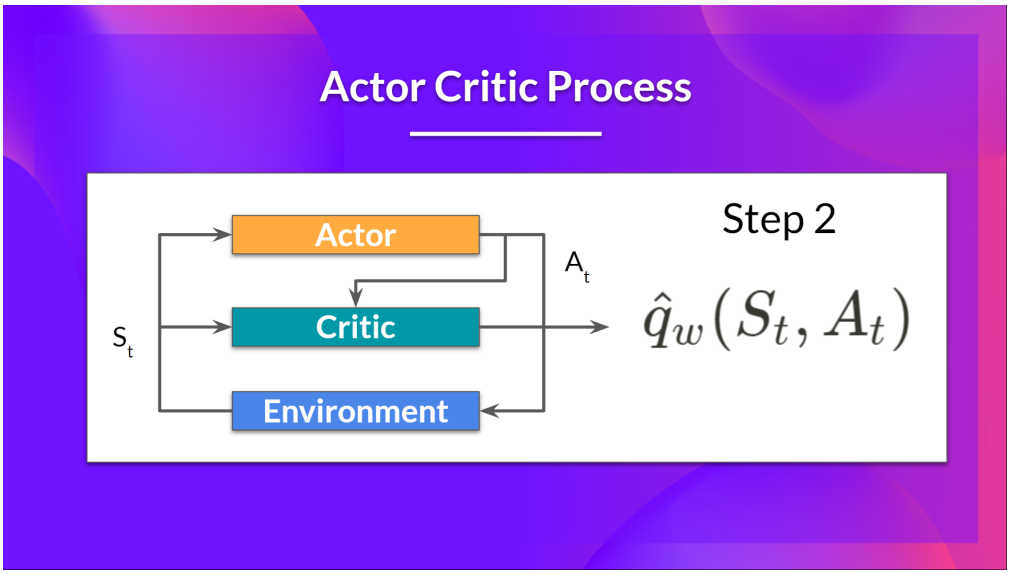


Figure 16: Actor-Critic process: critic evaluates the state-action pair [original figure]

Q5 solution

Correct answer(s): B, C

A. The Critic does not learn any function during the training process Explanation: Both the Actor and the Critic function parameters are updated during training time

B. The Actor learns a policy function, while the Critic learns a value function

C. It adds resistance to stochasticity and reduces high variance

Q6 solution

Instead of using directly the Action-Value function of the Critic as it is, we could use an `Advantage` function. The idea behind an `Advantage` function is that we calculate the relative advantage of an action compared to the others possible at a state, averaging them.

In other words: how taking that action at a state is better compared to the average value of the state

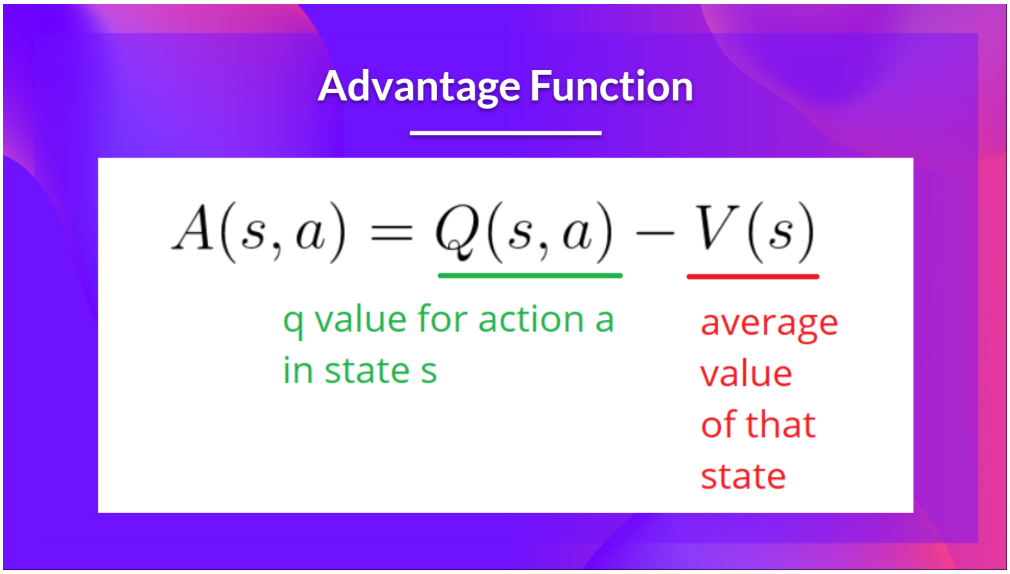


Figure 17: Definition of the advantage function [original figure]

5 Conclusion

Congrats on finishing this unit and the tutorial. You've just trained your first virtual robots .

Take time to grasp the material before continuing. You can also look at the additional reading materials we provided in the *additional reading* section.

Finally, we would love **to hear what you think of the course and how we can improve it**. If you have some feedback then please fill out this form

See you in next unit!

Keep learning, stay awesome

6 Additional Readings

6.1 Bias-variance tradeoff in Reinforcement Learning

If you want to dive deeper into the question of variance and bias tradeoff in Deep Reinforcement Learning, you can check out these two articles:

- Making Sense of the Bias / Variance Trade-off in (Deep) Reinforcement Learning
- Bias-variance Tradeoff in Reinforcement Learning

6.2 Advantage Functions

- Advantage Functions, SpinningUp RL

6.3 Actor Critic

- Foundations of Deep RL Series, L3 Policy Gradients and Advantage Estimation by Pieter Abbeel
- A2C Paper: Asynchronous Methods for Deep Reinforcement Learning